

```
#Now create a variable
vara = def_list

#Modification of vara will result in modifying def_list
vara.append("Hero")
print "Address of vara and def_list %s and %s"
"%(id(vara),id(def_list))", '\n\n'
print "vara = %s"%(vara), '\n\n'
print "def_list = %s"%(def_list), '\n\n'

# Define a simple tuple with multiple datatypes
def_tuple = 1,2,"1","100","Python","Anne","AI@345<_()","True,
False,{1:100,2:200,3:300},range(10)
# Print and view the tuple
print "Printing the tuple - ", def_tuple, '\n\n'
print "Printing the tuple's address - ", id(def_tuple), '\n\n'
# The address is subject to change
```

Once this is done, save the file with the .py extension. In this case, it is saved as *PythonPractice.py*. In the command prompt, browse to the location where the file is saved and execute the command *Python PythonPractice.py* (shown in Figure 4); this displays the result on the console (as shown in Figure 5).

Python is famous for its functional programming. A function is a block of organised, reusable code that is used to perform a task. It provides better modularity because of the ability to be reused.

```
1 # An empty dict
2 dict1 = dict() # same as dict1 = {}
3 dict2 = dict1 # same as dict2 = {}
4 # Adding values to dict
5 for i in [1,2,3,4,5,6,7]:
6     dict1[i] = i
7 # Adding values to a dict
8 # let us use a simple for loop to add values to a dict
9 for i in range(10):
10     dict1[i] = i * 10
11 print dict1, '\n\n', dict2, '\n\n'
12 ...
13 ... Define a simple list with multiple datatypes
14 def_list = [1,2,"1","100","Python","Anne","AI@345<_()","True,False,{1:100,2:200,3:300},range(10)]
15 # Now create a variable
16 vara = def_list
17 # Modification of vara will result in modifying def_list
18 vara.append("Hero")
19 print "Address of vara and def_list %s and %s"%(id(vara),id(def_list)), '\n\n'
20 print "vara = %s"%(vara), '\n\n'
21 print "def_list = %s"%(def_list), '\n\n'
22 # Define a simple tuple with multiple datatypes
23 def_tuple = 1,2,"1","100","Python","Anne","AI@345<_()","True,False,{1:100,2:200,3:300},range(10)
24 # Print and view the tuple
25 print "Printing the tuple - ", def_tuple, '\n\n'
26 print "Printing the tuple's address - ", id(def_tuple), '\n\n'
# The address is subject to change
```

Figure 4: A Python file to test the dictionary, list and tuples

```
2:\nanofthon-master\code\features\datatypes>Python PythonPractice.py
[0: 10: 11: 12: 13: 14: 15: 16: 17: 18: 19: 20: 21: 22: 23: 24: 25: 26: 27: 28: 29: 30: 31: 32: 33: 34: 35: 36: 37: 38: 39: 40: 41: 42: 43: 44: 45: 46: 47: 48: 49: 50: 51: 52: 53: 54: 55: 56: 57: 58: 59: 60: 61: 62: 63: 64: 65: 66: 67: 68: 69: 70: 71: 72: 73: 74: 75: 76: 77: 78: 79: 80: 81: 82: 83: 84: 85: 86: 87: 88: 89: 90: 91: 92: 93: 94: 95: 96: 97: 98: 99: 100]
Address of vara and def_list 36774216 and 36774216
vara = [1, 2, '1', '100', 'Python', 'Anne', 'AI@345<_()', True, False, {1: 100, 2: 200, 3: 300}, (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)]
def_list = [1, 2, '1', '100', 'Python', 'Anne', 'AI@345<_()', True, False, {1: 100, 2: 200, 3: 300}, (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)]
Printing the tuple - (1, 2, '1', '100', 'Python', 'Anne', 'AI@345<_()', True, False, {1: 100, 2: 200, 3: 300}, (0, 1, 2, 3, 4, 5, 6, 7, 8, 9))
Printing the tuple's address - 36161328
2:\nanofthon-master\code\features\datatypes>
```

Figure 5: Outcome of executing the *PythonPractice.py* file

There are a few rules for writing the function, which are:

- It begins with the 'def' keyword, followed by the name and parentheses, which can be used to pass the input parameters.
- A code block within every function starts with a colon (:) and is indented. A simple example of a function is shown below:

```
#A Function to print the sum of numbers
def funcA(a,b,c):
    #Add a,b,c and return the result
    sum_numbers = a + b + c
    return sum_numbers
```

This can be called by executing *print funcA(100,200,300)*, which prints the result 600 in the console.

There are many other useful features of Python like modules, inheritance, multi-processors, exceptions, file input and output, all of which make Python a high level and easy programming language. Python also has classes and objects, multi-threading, database access, networking, emails and many more features, which can be learned while exploring the advanced features. **END**

By: Ashish Kumar Sinha

The author is a software engineer based in Bengaluru. A software enthusiast at heart, he is passionate about using open source technology and sharing it with the world. He can be reached at ashi.sinha.87@gmail.com. Twitter handle: @sinha_tweet

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

Your favourite Magazine on Open Source is now on the Web, too.

OpenSourceForU.com

Follow us on Twitter @LinuxForYou